

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



BLM 4522 PROJE RAPORU

Ağ Tabanlı Paralel Dağıtım Sistemleri

Final 1. Proje

Veritabanı Performans Optimizasyonu ve İzleme Projesi

<https://github.com/zeyneph61/Ag-Tabanlı-Paralel-Dağıtım-Sistemleri-Projeleri>

<https://www.youtube.com/watch?v=kzXuyvZLGlo>

Zeynep Hacısalıhoğlu

22290449

İÇİNDEKİLER

İÇİNDEKİLER	3
1. GİRİŞ	4
1.1 Amaç ve Kapsam	4
1.2 Kullanılan Ortam.....	4
1.3 Veri Tabanı Kurulumu.....	4
2. Performans İzleme	5
2.1 DMV ile Sorgu Performansının İzlenmesi	5
2.2 SQL Profiler ile İzleme	6
2.3 Test Sorgusu Oluşturulması	8
2.4 Execution Plan Analizi	9
3. Index Yönetimi	10
3.1 Mevcut Indexlerin İncelenmesi	10
3.2 Composite Index Oluşturulması.....	11
3.3 Gereksiz Index Tespiti ve Silinmesi	12
4. Sorgu Optimizasyonu.....	13
4.1 Sorgunun İyileştirilmesi.....	13
4.2 Execution Plan Karşılaştırması	14
5. Disk Alanı Yönetimi	15
5.1 Veritabanı Dosya Boyutlarının İncelenmesi	15
5.2 Tablo Bazında Disk Kullanımı.....	16
5.3 SHRINK ile Alan Geri Kazanımı	17
6. Sonuç.....	18

1. GİRİŞ

Bu proje kapsamında Microsoft SQL Server 2022 üzerinde çalışan Northwind veritabanında performans analizi ve optimizasyon çalışmaları gerçekleştirilmiştir. Dynamic Management Views (DMV) aracılığıyla sistem izlenmiş, Execution Plan analiziyle darboğazlar tespit edilmiş; index yönetimi ve sorgu iyileştirme teknikleriyle bu sorunlar giderilmiştir.

1.1 Amaç ve Kapsam

Çalışmanın temel amacı veri tabanı sorgularını izleyerek darboğazları tespit etmek ve index yönetimi ile sorgu iyileştirme teknikleri aracılığıyla sistem performansını artırmaktır. Bu doğrultuda aşağıdaki adımlar uygulanmıştır:

- DMV kullanılarak mevcut sorgu performansı izlenmiştir.
- SQL Server Profiler ile gerçek zamanlı sorgu izleme gerçekleştirilmiştir.
- Test sorgusu oluşturularak Execution Plan analizi yapılmıştır.
- Mevcut indexler incelenmiş, eksik ve gereksiz indexler tespit edilmiştir.
- Composite index oluşturularak yüksek maliyetli Anahtar Arama operasyonu ortadan kaldırılmıştır.
- Duplicate indexler silinerek index yapısı sadeleştirilmiştir.
- Sorgu yapısı iyileştirilerek erken filtreleme sağlanmıştır.
- Disk alanı analizi yapılmış, SHRINK işlemiyle gereksiz alan geri kazanılmıştır.

1.2 Kullanılan Ortam

- Veri tabanı Sistemi: Microsoft SQL Server 2022 Developer Edition, Sürüm 16.0.1000.6
- Yönetim Aracı: SQL Server Management Studio (SSMS)

1.3 Veri Tabanı Kurulumu

Proje kapsamında kullanılacak örnek veri tabanı olarak Northwind seçilmiştir. Northwind, Microsoft tarafından yayımlanmış; bir ticaret şirketinin sipariş, ürün, müşteri ve çalışan verilerini barındıran klasik bir örnek veri tabanıdır. Veri tabanı SSMS üzerinden başarıyla yüklenmiş ve önceki projelerde doğrulanmıştır.

2. Performans İzleme

2.1 DMV ile Sorgu Performansının İzlenmesi

Sistemde çalışan sorguların performans verilerini incelemek amacıyla “sys.dm_exec_query_stats” ve “sys.dm_exec_sql_text” Dynamic Management View'ları kullanılmıştır. Bu görünüm; sorguların ortalama çalışma süresi, mantıksal okuma sayısı ve çalıştırılma sayısı gibi bilgileri sunmaktadır.

Aşağıdaki sorgu ile sisteme en fazla yük bindiren ilk 10 sorgu listelenmiştir:

```
SELECT TOP 10
    qs.total_elapsed_time / qs.execution_count AS avg_elapsed_time,
    qs.total_logical_reads / qs.execution_count AS avg_logical_reads,
    qs.execution_count,
    SUBSTRING(qt.text, (qs.statement_start_offset/2)+1,
        ((CASE qs.statement_end_offset
            WHEN -1 THEN DATALENGTH(qt.text)
            ELSE qs.statement_end_offset
        END - qs.statement_start_offset)/2)+1) AS query_text
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
ORDER BY avg_elapsed_time DESC;
```

Sorgu çalıştırıldığında dönen sonuçlar incelenmiş; listenin büyük bölümünün SSMS'in kendi iç sorgularından oluştuğu görülmüştür. Northwind veri tabanına henüz yeterli yük binmediğinden bu sonuç beklenen bir durumdur.

	avg_elapsed_time	avg_logical_reads	execution_count	query_text
1	164525	85789	1	SELECT SCHEMA_NAME(udf.schema_id) AS [Schema], u...
2	105176	113970	1	SELECT SCHEMA_NAME(v.schema_id) AS [Schema], v.na...
3	78155	81403	1	SELECT SCHEMA_NAME(sp.schema_id) AS [Schema], sp....
4	19329	210	1	SELECT udf.name AS [Name], udf.object_id AS [ID], udf.cre...
5	4349	224	1	SELECT @SmoAuditLevel AS [AuditLevel], ISNULL(@Num...
6	3830	388	1	SELECT log.name AS [Name], log.principal_id AS [ID], ISN...
7	3231	1201	1	SELECT ISNULL(actual_state, -2) FROM sys.database_qu...
8	2819	47	1	SELECT SCHEMA_NAME(xproc.schema_id) AS [Schema], ...
9	2189	65	1	SELECT tr.name AS [Name], tr.object_id AS [ID], CAST(...
10	1893	77	1	SELECT SCHEMA_NAME(obj.schema_id) AS [Schema], o...

2.2 SQL Profiler ile İzleme

DMV analizinin ardından SQL Server Profiler aracı kullanılarak veri tabanı üzerinde gerçek zamanlı sorgu izleme gerçekleştirilmiştir. Profiler; sorguların CPU kullanımı, okuma sayısı ve çalışma süresi gibi metriklerini canlı olarak takip etmeye imkân tanımaktadır.

İzleme başlatıldıktan sonra Northwind veri tabanı üzerinde üç farklı test sorgusu çalıştırılmıştır:

```
SELECT * FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
WHERE c.Country = 'Germany';

SELECT * FROM Products p
JOIN [Order Details] od ON p.ProductID = od.ProductID
WHERE p.UnitPrice > 20;

SELECT CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID
ORDER BY OrderCount DESC;
```

Profiler ekranında her üç sorgunun SQL:BatchCompleted olayı altında yakalandığı görülmüş; CPU, Reads ve Duration değerleri başarıyla raporlanmıştır.

Yakalanan sorgulara ait performans metrikleri incelendiğinde:

- *SELECT * FROM Orders JOIN Customers* — CPU: 15, Reads: 223, Duration: 4
- *SELECT * FROM Products JOIN Order Details* — CPU: 16, Reads: 502, Duration: 5
- *SELECT CustomerID, COUNT() FROM Orders* — CPU: 0, Reads: 344, Duration: 2*

Bu değerler sorgular arasındaki kaynak tüketimi farklarını ortaya koymaktadır. En fazla okuma işlemi Products ve Order Details tablolarını birleştiren sorguda gerçekleşmiştir.

1.Sorgu

```
SELECT * FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
WHERE c.Country = 'Germany';
```

İzleme çalışıyor.

2.Sorgu

```
SELECT * FROM Products p
JOIN [Order Details] od ON p.ProductID = od.ProductID
WHERE p.UnitPrice > 20;
```

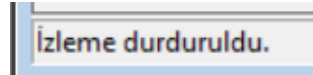
İzleme çalışıyor.

3.Sorgu

```
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID
ORDER BY OrderCount DESC;
```

İzleme çalışıyor.

İzleme tamamlandıktan sonra trace durdurulmuştur.



2.3 Test Sorgusu Oluşturulması

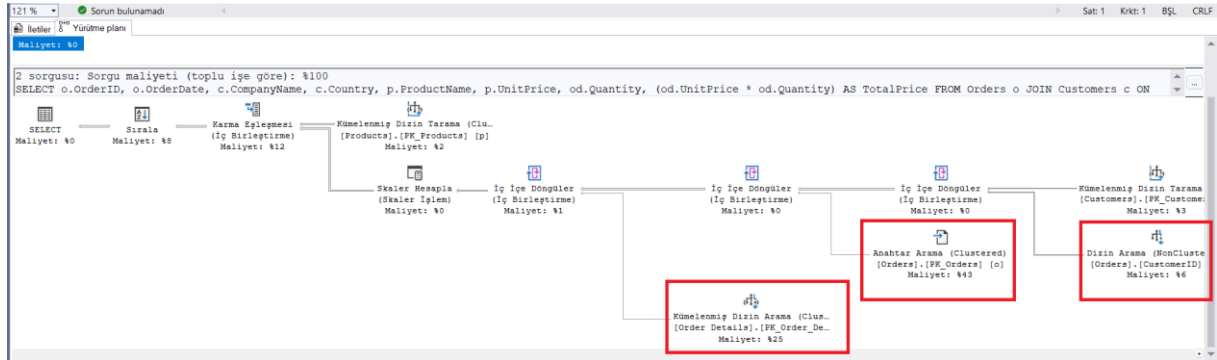
Northwind veritabanı üzerinde anlamlı performans verisi elde edebilmek için birden fazla tabloyu birleştiren ve filtreleme içeren bir test sorgusu oluşturulmuştur. Sorgu; Orders, Customers, Order Details ve Products tablolarını JOIN ile birleştirerek Almanya'ya ait siparişleri listelemektedir.

	OrderID	OrderDate	CompanyName	Country	ProductName	UnitPrice	Quantity	TotalPrice
1	10865	1998-02-02 00:00:00.000	QUICK-Stop	Germany	Côte de Blaye	263,50	60	15810,00
2	10540	1997-05-19 00:00:00.000	QUICK-Stop	Germany	Côte de Blaye	263,50	30	7905,00
3	10817	1998-01-06 00:00:00.000	Königlich Essen	Germany	Côte de Blaye	263,50	30	7905,00
4	10515	1997-04-23 00:00:00.000	QUICK-Stop	Germany	Schoggi Schokolade	43,90	120	5268,00
5	10691	1997-10-03 00:00:00.000	QUICK-Stop	Germany	Thüringer Rostbratwurst	123,79	40	4951,60
6	10267	1996-07-29 00:00:00.000	Frankenversand	Germany	Raclette Courdavault	55,00	70	3080,00
7	10893	1998-02-18 00:00:00.000	Königlich Essen	Germany	Thüringer Rostbratwurst	123,79	24	2970,96
8	10515	1997-04-23 00:00:00.000	QUICK-Stop	Germany	Camembert Pierrot	34,00	84	2856,00
9	10694	1997-10-06 00:00:00.000	QUICK-Stop	Germany	Uncle Bob's Organic Dried Pears	30,00	90	2700,00
10	10549	1997-05-27 00:00:00.000	QUICK-Stop	Germany	Manjimup Dried Apples	53,00	48	2544,00
11	10588	1997-07-03 00:00:00.000	QUICK-Stop	Germany	Carnarvon Tigers	62,50	40	2500,00
12	10745	1997-11-18 00:00:00.000	QUICK-Stop	Germany	Raclette Courdavault	55,00	45	2475,00
13	10691	1997-10-03 00:00:00.000	QUICK-Stop	Germany	Tarte au sucre	49,30	48	2366,40
14	11021	1998-04-14 00:00:00.000	QUICK-Stop	Germany	Manjimup Dried Apples	53,00	44	2332,00
15	10451	1997-02-19 00:00:00.000	QUICK-Stop	Germany	Pâté chinois	24,00	120	2304,00
16	10345	1996-11-04 00:00:00.000	QUICK-Stop	Germany	Northwoods Cranberry Sauce	40,00	70	2240,00
17	10772	1997-12-10 00:00:00.000	Lehmanns Marktstand	Germany	Thüringer Rostbratwurst	123,79	18	2228,22
18	11021	1998-04-14 00:00:00.000	QUICK-Stop	Germany	Gumbär Gummibärchen	31,23	63	1967,49
19	10658	1997-09-05 00:00:00.000	QUICK-Stop	Germany	Camembert Pierrot	34,00	55	1870,00
20	10691	1997-10-03 00:00:00.000	QUICK-Stop	Germany	Ipoh Coffee	46,00	40	1840,00
21	10791	1997-12-23 00:00:00.000	Frankenversand	Germany	Thüringer Rostbratwurst	123,79	14	1733,06
22	10593	1997-07-09 00:00:00.000	Lehmanns Marktstand	Germany	Sir Rodney's Marmalade	81,00	21	1701,00
23	10249	1996-07-05 00:00:00.000	Toms Spezialitäten	Germany	Manjimup Dried Apples	53,00	40	1696,00
24	10765	1997-12-04 00:00:00.000	QUICK-Stop	Germany	Louisiana Fiery Hot Pepper Sa...	21,05	80	1684,00

2.4 Execution Plan Analizi

Sorgunun kaynak kullanımını ayrıntılı incelemek amacıyla SSMS üzerinden Actual Execution Plan etkinleştirilmiş ve sorgu tekrar çalıştırılmıştır. Plan incelendiğinde aşağıdaki kritik operasyonlar tespit edilmiştir:

- Anahtar Arama (Clustered) — Maliyet: %43: Orders tablosunda mevcut index ihtiyaç duyulan tüm kolonları karşılamadığından SQL Server ana tabloya geri dönmek zorunda kalmaktadır. Bu operasyon planın en pahalı adımını oluşturmaktadır.
- Kümelenmiş Dizin Tarama — Maliyet: %25: Order Details tablosunda index araması yerine tam tarama yapılmaktadır.
- Dizin Arama (NonClustered) — Maliyet: %6: Orders tablosundaki CustomerID indexi kullanılmış ancak tek başına yeterli olmamıştır.



3. Index Yönetimi

3.1 Mevcut Indexlerin İncelenmesi

Execution Plan analizinin ardından Orders, Customers, Order Details ve Products tablolarındaki mevcut index yapısı sorgulanmıştır.

```
SELECT
    t.name AS table_name,
    i.name AS index_name,
    i.type_desc,
    STRING_AGG(c.name, ', ') AS columns
FROM sys.indexes i
JOIN sys.index_columns ic ON i.object_id = ic.object_id AND i.index_id = ic.index_id
JOIN sys.columns c ON ic.object_id = c.object_id AND ic.column_id = c.column_id
JOIN sys.tables t ON i.object_id = t.object_id
WHERE t.name IN ('Orders', 'Customers', 'Order Details', 'Products')
GROUP BY t.name, i.name, i.type_desc
ORDER BY t.name, i.name;
```

Sonuçlar incelendiğinde Orders tablosunda CustomerID ve CustomersOrders adlı iki ayrı indexin aynı kolonu kapsadığı, benzer şekilde EmployeeID ve EmployeesOrders indexlerinin de birbirini tekrar ettiği tespit edilmiştir.

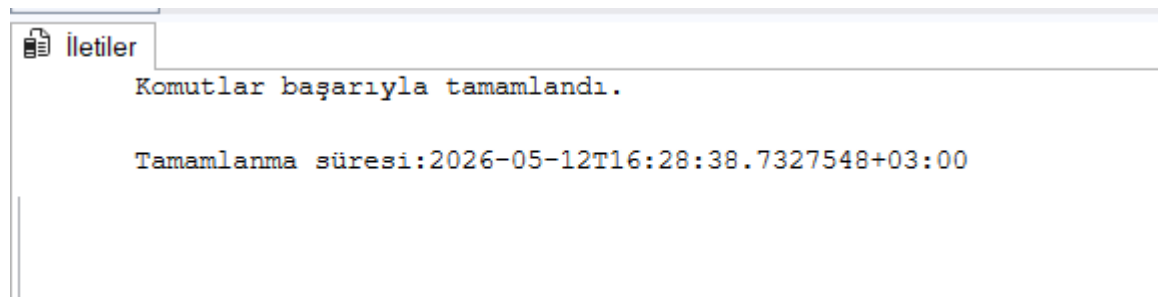
	table_name	index_name	type_desc	columns
1	Customers	City	NONCLUSTERED	City
2	Customers	CompanyName	NONCLUSTERED	CompanyName
3	Customers	PK_Customers	CLUSTERED	CustomerID
4	Customers	PostalCode	NONCLUSTERED	PostalCode
5	Customers	Region	NONCLUSTERED	Region
6	Order Details	OrderID	NONCLUSTERED	OrderID
7	Order Details	OrdersOrder_Details	NONCLUSTERED	OrderID
8	Order Details	PK_Order_Details	CLUSTERED	OrderID, ProductID
9	Order Details	ProductID	NONCLUSTERED	ProductID
10	Order Details	ProductsOrder_Details	NONCLUSTERED	ProductID
11	Orders	CustomerID	NONCLUSTERED	CustomerID
12	Orders	CustomersOrders	NONCLUSTERED	CustomerID
13	Orders	EmployeeID	NONCLUSTERED	EmployeeID
14	Orders	EmployeesOrders	NONCLUSTERED	EmployeeID
15	Orders	OrderDate	NONCLUSTERED	OrderDate
16	Orders	PK_Orders	CLUSTERED	OrderID
17	Orders	ShippedDate	NONCLUSTERED	ShippedDate
18	Orders	ShippersOrders	NONCLUSTERED	ShipVia
19	Orders	ShipPostalCode	NONCLUSTERED	ShipPostalCode
20	Products	CategoriesProducts	NONCLUSTERED	CategoryID
21	Products	CategoryID	NONCLUSTERED	CategoryID
22	Products	PK_Products	CLUSTERED	ProductID
23	Products	ProductName	NONCLUSTERED	ProductName
24	Products	SupplierID	NONCLUSTERED	SupplierID
25	Products	SuppliersProducts	NONCLUSTERED	SupplierID

3.2 Composite Index Oluşturulması

Execution Plan'da %43 maliyet üreten Anahtar Arama operasyonunu ortadan kaldırmak amacıyla CustomerID ve OrderDate kolonlarını birleştiren, sık kullanılan diğer kolonları INCLUDE eden bir composite index oluşturulmuştur.

```
CREATE NONCLUSTERED INDEX IX_Orders_CustomerID_OrderDate  
ON Orders (CustomerID, OrderDate)  
INCLUDE (OrderID, EmployeeID, ShipVia);
```

Bu sayede SQL Server, sorgu sırasında ihtiyaç duyduğu tüm kolonlara doğrudan index üzerinden erişebilmekte; ana tabloya geri dönme ihtiyacı ortadan kalkmaktadır.



3.3 Gereksiz Index Tespiti ve Silinmesi

Mevcut index listesi incelendiğinde Orders tablosunda birbirini tekrar eden indexler tespit edilmiştir:

- CustomerID ve CustomersOrders → her ikisi de yalnızca CustomerID kolonunu kapsamaktadır.
- EmployeeID ve EmployeesOrders → her ikisi de yalnızca EmployeeID kolonunu kapsamaktadır.

Duplicate indexler gereksiz disk alanı tüketmekte ve her INSERT/UPDATE işleminde ek maliyet oluşturmaktadır. Bu nedenle tekrar eden indexler aşağıdaki sorgu ile silinmiştir:

```
DROP INDEX CustomersOrders ON Orders;  
DROP INDEX EmployeesOrders ON Orders;
```

Silme işleminin ardından index listesi tekrar sorgulanmış ve Orders tablosundaki index sayısının 10'dan 8'e düştüğü doğrulanmıştır.

	table_name	index_name	index_columns
1	Orders	CustomerID	CustomerID
2	Orders	CustomersOrders	CustomerID
3	Orders	EmployeeID	EmployeeID
4	Orders	EmployeesOrders	EmployeeID
5	Orders	IX_Orders_CustomerID_OrderDate	CustomerID, OrderDate
6	Orders	OrderDate	OrderDate
7	Orders	PK_Orders	OrderID
8	Orders	ShippedDate	ShippedDate
9	Orders	ShippersOrders	ShipVia
10	Orders	ShipPostalCode	ShipPostalCode

	table_name	index_name	index_columns
1	Orders	CustomerID	CustomerID
2	Orders	EmployeeID	EmployeeID
3	Orders	IX_Orders_CustomerID_OrderDate	CustomerID, OrderDate
4	Orders	OrderDate	OrderDate
5	Orders	PK_Orders	OrderID
6	Orders	ShippedDate	ShippedDate
7	Orders	ShippersOrders	ShipVia
8	Orders	ShipPostalCode	ShipPostalCode

4. Sorgu Optimizasyonu

4.1 Sorgunun İyileştirilmesi

Test aşamasında kullanılan orijinal sorguda WHERE koşulu içinde yer alan

c.Country = 'Germany' filtresi, JOIN işlemleri tamamlandıktan sonra uygulanmaktadır.

Bu durum SQL Server'ın gereksiz satırları işlemesine neden olmaktadır.

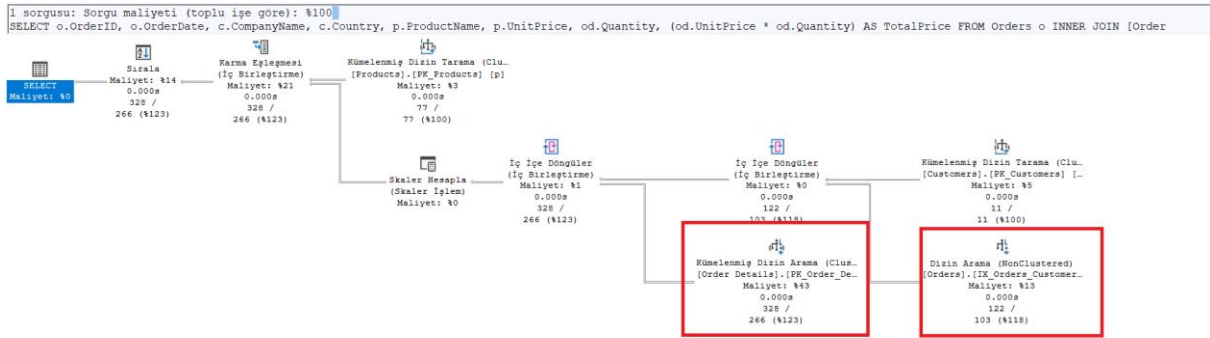
Filtrenin JOIN koşuluna taşınmasıyla SQL Server, Customers tablosunu daha erken aşamada filtreleyerek sonraki JOIN işlemlerini daha az satır üzerinde gerçekleştirebilmektedir.

```
SELECT o.OrderID, o.OrderDate, c.CompanyName, c.Country,
       p.ProductName, p.UnitPrice, od.Quantity,
       (od.UnitPrice * od.Quantity) AS TotalPrice
FROM Orders o
INNER JOIN [Order Details] od ON o.OrderID = od.OrderID
INNER JOIN Customers c ON o.CustomerID = c.CustomerID AND c.Country = 'Germany'
INNER JOIN Products p ON od.ProductID = p.ProductID
WHERE o.OrderDate BETWEEN '1996-01-01' AND '1998-12-31'
ORDER BY TotalPrice DESC;
```

4.2 Execution Plan Karşılaştırması

Optimize edilmiş sorgunun Execution Plan'ı incelendiğinde Anahtar Arama operasyonunun tamamen ortadan kalktığı görülmüştür. Composite index sayesinde SQL Server artık ihtiyaç duyduğu tüm kolonlara doğrudan index üzerinden erişmektedir.

Operasyon	Öncesi	Sonrası
Anahtar Arama (Clustered)	%43	Yok
Dizin Arama (NonClustered)	%6	Doğrudan kullanıyor
Genel plan karmaşıklığı	Yüksek	Düşük



5. Disk Alanı Yönetimi

5.1 Veri tabanı Dosya Boyutlarının İncelenmesi

Veri tabanının disk üzerinde kapladığı alanı incelemek amacıyla sys.database_files görünümü sorgulanmıştır.

```
SELECT
    name AS file_name,
    type_desc,
    physical_name,
    size * 8 / 1024 AS size_mb,
    max_size,
    growth
FROM sys.database_files;
```

Sorgu sonucunda Northwind veri dosyasının **72 MB**, log dosyasının ise **8 MB** disk alanı kapladığı görülmüştür. Growth değerinin 8192 olması, her otomatik büyümede dosyanın 64 MB genişleyeceğini göstermektedir.

	file_name	type_desc	physical_name	size_mb	max_size	growth
1	Northwind	ROWS	C:\Program Files\Microsoft SQL Server\MSSQL16.DB\...	72	-1	8192
2	Northwind_log	LOG	C:\Program Files\Microsoft SQL Server\MSSQL16.DB\...	8	268435456	8192

5.2 Tablo Bazında Disk Kullanımı

Hangi tabloların ne kadar alan kullandığını tespit etmek amacıyla aşağıdaki sorgu çalıştırılmıştır.

```
SELECT
    t.name AS table_name,
    p.rows AS row_count,
    SUM(a.total_pages) * 8 AS total_kb,
    SUM(a.used_pages) * 8 AS used_kb,
    (SUM(a.total_pages) - SUM(a.used_pages)) * 8 AS unused_kb
FROM sys.tables t
JOIN sys.indexes i ON t.object_id = i.object_id
JOIN sys.partitions p ON i.object_id = p.object_id AND i.index_id = p.index_id
JOIN sys.allocation_units a ON p.partition_id = a.container_id
GROUP BY t.name, p.rows
ORDER BY total_kb DESC;
```

Sonuçlar incelendiğinde Orders tablosunun toplam alanının yaklaşık %45'inin, Employees tablosunun ise büyük bölümünün kullanılmadığı tespit edilmiştir.

	table_name	row_count	total_kb	used_kb	unused_kb
1	Orders	830	832	456	376
2	Employees	9	480	272	208
3	Products	77	432	96	336
4	Order Details	2155	424	280	144
5	Customers	92	360	104	256
6	Suppliers	29	288	64	224
7	Categories	8	280	136	144
8	Region	4	144	32	112
9	EmployeeTerritories	49	144	32	112
10	Territories	53	144	32	112
11	PITRTest	1	72	16	56
12	Shippers	3	72	16	56
13	TestSatis	3	72	16	56
14	CustomerCustomerDemo	0	0	0	0
15	CustomerDemographics	0	0	0	0

5.3 SHRINK ile Alan Geri Kazanımı

Kullanılmayan disk alanını geri kazanmak amacıyla DBCC SHRINKDATABASE komutu çalıştırılmıştır.

```
DBCC SHRINKDATABASE (Northwind, 10);

SELECT
    name AS file_name,
    size * 8 / 1024 AS size_mb
FROM sys.database_files;
```

İşlem sonucunda Northwind veri dosyasının boyutu 72 MB'dan 9 MB'a düşmüştür. Bu işlem sayesinde veri tabanının kapladığı alan yaklaşık %87 oranında azaltılmıştır.

	DblId	FileId	CurrentSize	MinimumSize	UsedPages	EstimatedPages
1	9	1	1224	1024	1080	1080

	file_name	size_mb
1	Northwind	9
2	Northwind_log	8

6. Sonuç

Bu proje kapsamında Northwind veri tabanı üzerinde performans analizi ve optimizasyon çalışmaları gerçekleştirilmiştir. Yapılan çalışmalar sonucunda aşağıdaki iyileştirmeler sağlanmıştır:

- DMV kullanılarak sistem genelinde sorgu performansı izlenmiş ve analiz edilmiştir.
- SQL Server Profiler ile gerçek zamanlı sorgu izleme gerçekleştirilmiş; CPU, okuma sayısı ve süre metrikleri takip edilmiştir.
- Execution Plan incelemesiyle %43 maliyet üreten Anahtar Arama operasyonu tespit edilmiştir.
- Oluşturulan composite index sayesinde bu operasyon tamamen ortadan kaldırılmıştır.
- Duplicate indexlerin silinmesiyle Orders tablosundaki index yapısı sadeleştirilmiş, gereksiz kaynak tüketimi önlenmiştir.
- Sorgu yapısının iyileştirilmesiyle erken filtreleme sağlanmış ve SQL Server'ın daha az satır üzerinde çalışması mümkün kılınmıştır.
- Disk alanı analizi yapılmış, SHRINK işlemiyle veritabanı boyutu 72 MB'dan 9 MB'a düşürülmüştür.

Çalışma boyunca SQL Server'ın sunduğu DMV'ler, Profiler ve Execution Plan araçlarının performans sorunlarının tespitinde ne denli etkili olduğu görülmüştür.